

1. Descripción del problema

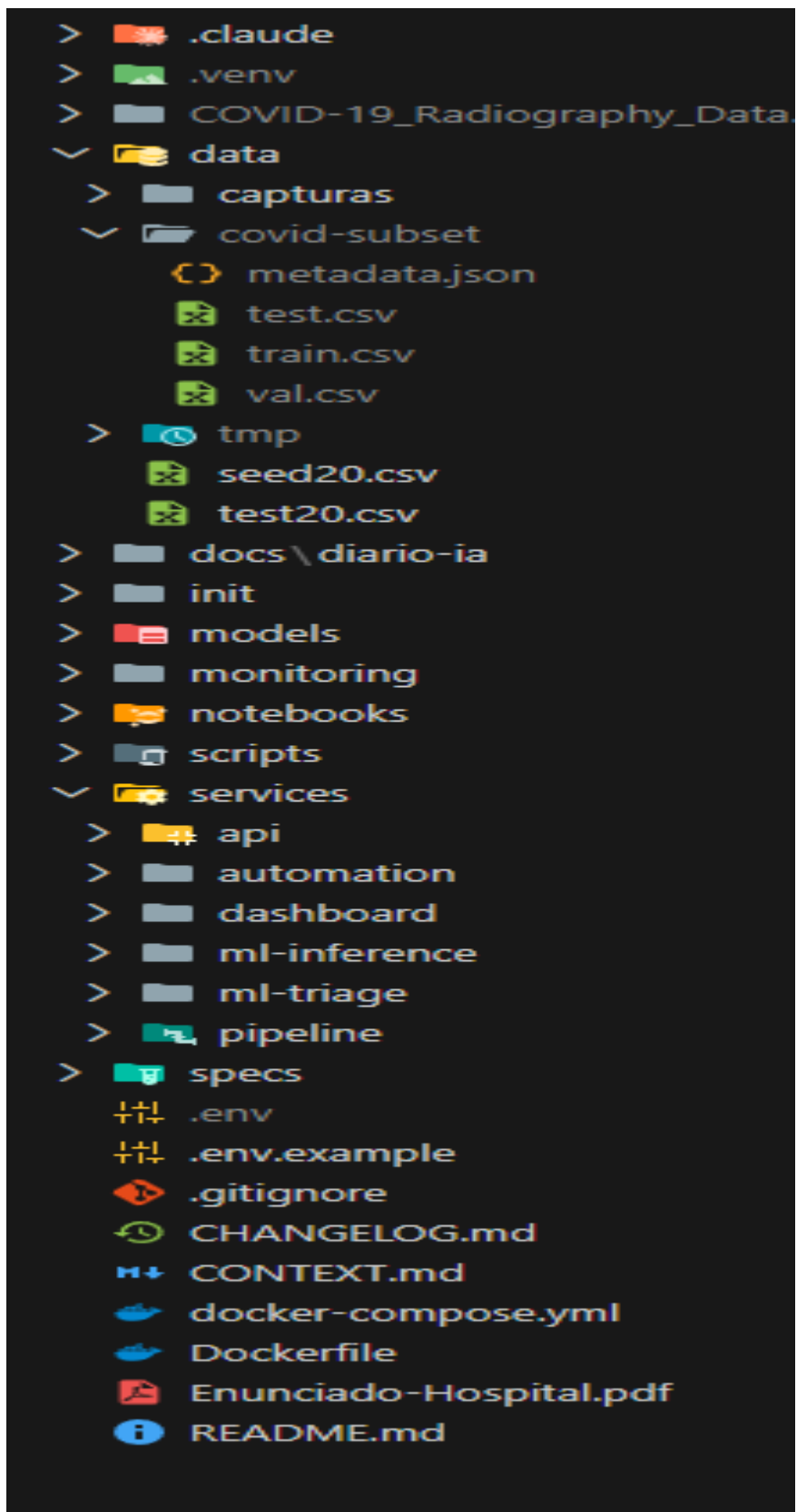
1.1 Contexto del proyecto

El hospital laSalle Health Center se plantea como una organización sanitaria de tamaño medio en proceso de transformación digital. En este contexto, hemos diseñado e implementado una solución basada en Inteligencia Artificial y Big Data para mejorar la gestión de datos clínicos y operativos. El punto de partida es un escenario realista: el hospital dispone de datos generados diariamente, como fichas de pacientes, síntomas, registros de ingresos, pruebas diagnósticas, radiografías y logs de sistemas, pero necesita una infraestructura que convierta esos datos en información útil para la toma de decisiones.

Nuestra propuesta simula un sistema hospitalario completo, no únicamente un modelo aislado. Por ese motivo integramos ingesta de datos, almacenamiento combinado, procesamiento escalable, modelos de IA, API, dashboard, monitorización, validación de calidad y alertas operativas.

1.2 Objetivos planteados

- Diseñar una arquitectura containerizada y reproducible mediante Docker Compose.
- Implementar un pipeline de datos completo con ingesta, validación, transformación, almacenamiento y servicio.
- Utilizar almacenamiento combinado: MinIO/S3 para datos raw, PostgreSQL para datos estructurados y MongoDB para documentos, eventos y predicciones.
- Aplicar procesamiento escalable con Dask para el pipeline batch.
- Entrenar y desplegar modelos tabulares para triaje y sospecha de enfermedad.
- Entrenar y comparar modelos de Deep Learning para clasificación de radiografías de tórax.
- Exponer resultados mediante una API REST y un dashboard Streamlit.
- Centralizar logs con Loki/Promtail y crear alertas mediante el servicio automation.
- Documentar el proceso usando metodología SDD y diario de desarrollo asistido por IA.



2. Datos

2.1 Fuentes utilizadas

Hemos trabajado con tres grupos principales de datos: datos sintéticos tabulares de pacientes, ficheros CSV batch para el pipeline, y el dataset público COVID-19 Radiography Dataset para radiografías de tórax. La decisión de usar datos sintéticos y públicos responde a una restricción ética y legal: no debemos utilizar datos clínicos reales identificables en una práctica académica.

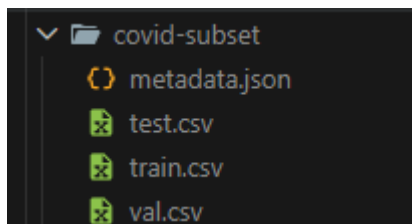
Fuente	Tipo	Uso en el proyecto	Motivo
Fichas sintéticas de pacientes	Tabular	Entrenamiento y pruebas del triaje.	Evita datos personales reales y permite reproducibilidad.
CSV batch generados por seed	Tabular / batch	Ingesta, validación y procesamiento por pipeline.	Permite demostrar ingesta automatizada y calidad de datos.
quality-test-invalid.csv	CSV inválido controlado	Prueba de validación y rechazos.	Demuestra detección de datos incompletos/corruptos.
COVID-19 Radiography Dataset	Imágenes médicas	Entrenamiento de clasificación de radiografías.	Dataset público con clases COVID, Normal, Viral Pneumonia y Lung_Opacity.
Logs de servicios	Eventos / texto	Monitorización y alertas.	Permite auditar la ejecución del sistema.

2.2 Proceso de limpieza y transformación

En el pipeline tabular aplicamos reglas declarativas de validación definidas en `validation_rules.yaml`. Estas reglas comprueban campos obligatorios, rangos, tipos, valores permitidos y coherencia entre campos. Cuando una fila no cumple los criterios, no se elimina silenciosamente: se persiste en MongoDB en la colección `ingestion_rejects` con el motivo del rechazo.

En radiografías detectamos una fuente potencial de contaminación: el dataset contiene carpetas `images/` y `masks/`. Las máscaras no son radiografías originales, sino segmentaciones binarias. Por ello corregimos `prepare_data.py` para leer exclusivamente desde `images/` y excluir `masks/`. Después generamos splits estratificados `train/val/test` y balanceamos las clases finales: Sana, Neumonía y COVID-19.

Clase original	Clase final	Justificación
Normal	Sana	Radiografías sin patología detectable.
Viral Pneumonia	Neumonía	Neumonía viral estándar.
Lung_Opacity	Neumonía	Agrupación necesaria para cumplir clasificación triple.
COVID	COVID-19	Patrones asociados a COVID-19.



3. Arquitectura del sistema

3.1 Diseño del pipeline de datos

El pipeline se ha diseñado como un flujo modular. No lo hemos implementado como un script único, sino como un conjunto de fases: ingesta, lectura escalable, validación, transformación, carga, inferencia, agregados y eventos. Esta separación permite probar cada parte y explicar claramente el flujo de datos.

Fase	Archivo principal	Entrada	Salida
Ingesta	ingestion.py	CSV raw desde MinIO/S3	DataFrame crudo y evento pipeline.file.read
Lectura escalable	scalable_csv.py / dask_runtime.py	CSV potencialmente grande	Lectura particionada con Dask
Validación	validation.py + validation_rules.yaml	DataFrame crudo	Registros válidos y rechazados
Transformación	transformation.py	Registros válidos	Entidades paciente e ingreso
Carga	loading.py	Entidades transformadas	PostgreSQL + MongoDB
Inferencia tabular	triage_client.py	Ficha validada	Triaje y sospecha de enfermedad
Agregados	aggregates.py	Resumen de ejecución	Métricas diarias
Eventos	events.py	Estado de fases	system_events en MongoDB

3.2 Infraestructura utilizada

Toda la infraestructura se levanta con Docker Compose. Esto permite que el sistema completo pueda arrancarse con un único comando y que cada responsabilidad esté aislada en su propio contenedor.

Servicio	Responsabilidad
api	Exposición REST para pacientes, batch runs, eventos y PDF.
dashboard	Interfaz Streamlit para pacientes, triaje, radiografías y estado.
pipeline	Orquestación batch/online del flujo de datos.
ml-triage	Inferencia tabular de triaje y sospecha de enfermedad.
ml-inference	Inferencia Deep Learning de radiografías.
postgres	Datos estructurados validados.
mongodb	Eventos, predicciones, rechazos, alertas y agregados.
minio	Landing zone S3-compatible para datos raw.
dask-scheduler	Planificación de tareas Dask.
dask-worker	Ejecución de tareas Dask.
loki	Centralización de logs.
promtail	Recolector de logs Docker hacia Loki.
automation	Escaneo de eventos warning/error y creación de alertas.

```
(.venv) PS C:\Users\aripa\Downloads\Practica_Hospital_BACKUP_20260516_145943> docker compose -p practica_hospital_ariadnascualpolballarin ps
```

NAME	SERVICE	CREATED	STATUS	IMAGE	PORTS	COMMAND
practica_hospital_ariadnascualpolballarin-api-1	api	3 days ago	Up 3 days (healthy)	practica_hospital_ariadnascualpolballarin-api	0.0.0.0:8005->8000/tcp, [::]:8005->8000/tcp	"uvicorn app:app..."
practica_hospital_ariadnascualpolballarin-automation-1	automation	3 days ago	Up 3 days	practica_hospital_ariadnascualpolballarin-automation		"python -u main.py"
practica_hospital_ariadnascualpolballarin-dashboard-1	dashboard	2 days ago	Up 2 days (healthy)	practica_hospital_ariadnascualpolballarin-dashboard	0.0.0.0:8502->8501/tcp, [::]:8502->8501/tcp	"streamlit run app.py"
practica_hospital_ariadnascualpolballarin-dask-scheduler-1	dask-scheduler	3 days ago	Up 3 days (healthy)	practica_hospital_ariadnascualpolballarin-dask-scheduler	0.0.0.0:8787->8787/tcp, [::]:8787->8787/tcp	"dask-scheduler --host 0.0.0.0"
practica_hospital_ariadnascualpolballarin-dask-worker-1	dask-worker	3 days ago	Up 3 days	practica_hospital_ariadnascualpolballarin-dask-worker		"dask-worker --dask-tcp://0.0.0.0:8787"
practica_hospital_ariadnascualpolballarin-loki-1	loki	3 days ago	Up 3 days (healthy)	grafana/loki:3.2.1	0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp	"./usr/bin/loki --config=/etc/loki-config.yaml"
practica_hospital_ariadnascualpolballarin-minio-1	minio	3 days ago	Up 3 days (healthy)	minio/minio:RELEASE.2024-12-18T13-15-44Z	0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp	"./usr/bin/minio server /data --console-address :9001"
practica_hospital_ariadnascualpolballarin-ml-inference-1	ml-inference	2 days ago	Up 2 days (healthy)	practica_hospital_ariadnascualpolballarin-ml-inference	0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp	"uvicorn app:app..."
practica_hospital_ariadnascualpolballarin-ml-triage-1	ml-triage	3 days ago	Up 3 days (healthy)	practica_hospital_ariadnascualpolballarin-ml-triage	8002/tcp	"uvicorn app:app..."
practica_hospital_ariadnascualpolballarin-mongo-express-1	mongo-express	3 days ago	Up 3 days (unhealthy)	mongo-express:1.0.2	0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp	"./sbin/docker-entrypoint.sh"
practica_hospital_ariadnascualpolballarin-mongodb-1	mongodb	3 days ago	Up 3 days (healthy)	mongo:8.0	27017/tcp	"docker-entrypoint.sh"
practica_hospital_ariadnascualpolballarin-pgadmin-1	pgadmin	3 days ago	Up 3 days (unhealthy)	dpagadmin/pgadmin4:9	0.0.0.0:5050->80/tcp, [::]:5050->80/tcp	"./entrypoint.sh"
practica_hospital_ariadnascualpolballarin-pipeline-1	pipeline	3 days ago	Up 3 days	practica_hospital_ariadnascualpolballarin-pipeline		"sleep infinity"
practica_hospital_ariadnascualpolballarin-postgres-1	postgres	3 days ago	Up 3 days (healthy)	postgres:17-alpine	5432/tcp	"docker-entrypoint.sh"
practica_hospital_ariadnascualpolballarin-promtail-1	promtail	3 days ago	Up 3 days	grafana/promtail:3.2.1		"./usr/bin/promtail -c /etc/promtail.yaml"

```
(.venv) PS C:\Users\aripa\Downloads\Practica_Hospital_BACKUP_20260516_145943>
```

3.3 Relación entre componentes

El dashboard se comunica con la API. La API lanza operaciones del pipeline o consulta datos ya procesados. El pipeline lee datos raw desde MinIO, valida y transforma, guarda datos estructurados en PostgreSQL, guarda eventos y documentos flexibles en MongoDB, y llama al servicio ml-triage para obtener predicciones tabulares. Para radiografías, el dashboard puede llamar al servicio ml-inference, que devuelve la clasificación de la imagen. Los logs de los contenedores son recogidos por Promtail y enviados a Loki; además, los eventos funcionales quedan en MongoDB y automation genera alertas.

4. Modelos de Inteligencia Artificial

4.1 Modelo tabular de triaje

El modelo de triaje utiliza fichas clínicas estructuradas con síntomas auto-reportados: edad, sexo, enfermedades crónicas, motivo principal, duración de síntomas, intensidad del dolor, fiebre, dificultad respiratoria, tos y contacto COVID. Su salida principal es una clase de prioridad: Alta, Media o Baja. La lógica de entrenamiento se basa en datos sintéticos reproducibles y reglas clínicas simuladas. Esta decisión es adecuada para una práctica académica porque evita el uso de datos clínicos reales y permite validar el flujo completo.

4.2 Modelo de sospecha de enfermedad

Además del triaje, entrenamos un modelo de sospecha de enfermedad. Este modelo no emite un diagnóstico definitivo, sino una orientación basada en síntomas, por ejemplo sospecha de COVID-19, sospecha de neumonía, cardiopatía aguda, gastroenteritis, traumatismo o cuadro inespecífico. En el dashboard se presenta como apoyo a la decisión y no como diagnóstico médico.

4.3 Modelo Deep Learning de radiografías

Para la clasificación de radiografías comparamos tres arquitecturas convolucionales: una CNN simple propia, ResNet18 y EfficientNet-B0. La CNN simple actúa como baseline entrenado desde cero. ResNet18 y EfficientNet-B0 son arquitecturas estándar disponibles en torchvision.models, usadas con transfer learning. No son servicios externos ni APIs: son implementaciones de redes neuronales convolucionales dentro de PyTorch/Torchvision. En ambos casos partimos de pesos preentrenados en ImageNet y sustituimos la cabeza final para clasificar tres clases: Sana, Neumonía y COVID-19.

El modelo no usa el nombre del archivo para decidir. Durante entrenamiento el filepath solo sirve para abrir la imagen, y la etiqueta se toma desde class_target. Durante inferencia, el servicio recibe bytes de imagen, los convierte a RGB, redimensiona a 224x224, normaliza y pasa el tensor al modelo. Por tanto, la predicción depende de los píxeles, no del nombre del fichero.

4.4 Proceso de entrenamiento

- Preparamos el dataset leyendo únicamente carpetas images/ y excluyendo masks/.
- Generamos splits train, validation y test estratificados.
- Entrenamos la CNN simple desde cero como baseline.
- Entrenamos ResNet18 con transfer learning.
- Entrenamos EfficientNet-B0 con transfer learning.
- Evaluamos todos los modelos con test.csv.
- Generamos metrics.json, confusion_matrix.png y critical_analysis.md para cada modelo.
- Generamos comparison.csv, comparison.md y comparison.json para comparar arquitecturas.

4.5 Evaluación mediante métricas y resultados obtenidos

No seleccionamos el modelo solo por accuracy. En contexto sanitario nos interesan especialmente F1 macro, recall COVID-19 y recall Neumonía, porque los falsos negativos pueden tener impacto clínico y epidemiológico.

Modelo	Accuracy	F1 macro	Recall COVID-19	Recall Neumonía	Decisión
EfficientNet-B0	0.9600	0.9601	0.9800	0.9333	Modelo seleccionado.
ResNet18	0.8933	0.8929	0.9667	0.8333	Buen recall COVID, peor neumonía.
Simple CNN	0.7022	0.7013	0.7000	0.7733	Baseline insuficiente.

Seleccionamos EfficientNet-B0 porque obtuvo el mejor equilibrio clínico: mayor F1 macro, mayor recall COVID-19 y mayor recall Neumonía. La comparación con CNN simple justificó que el transfer learning aportaba una mejora clara.

8. Comparación final de modelos

La comparación final se genera con `training.compare_models`.

El criterio de selección no es solo `accuracy`.

En contexto sanitario se prioriza:

- `f1_macro`;
- `recall_covid`;
- `recall_neumonia`;
- matriz de confusión;
- impacto clínico de falsos negativos.

	version	backbone	accuracy	f1_macro	recall_covid	recall_neumonia	best_val_f1_macro	test_rows	artifact_dir
0	rx-efficientnetb0-20260515-2b92eec9	efficientnet_b0	0.960000	0.960079	0.980000	0.933333	0.888084	450	/app/models/radiography/rx-efficientnetb0-2026...
1	rx-resnet18-20260515-d397f8e5	resnet18	0.893333	0.892919	0.966667	0.833333	0.892065	450	/app/models/radiography/rx-resnet18-20260515-d...
2	rx-simplecnn-20260515-e8ac76f0	simple_cnn	0.702222	0.701260	0.700000	0.773333	0.676096	450	/app/models/radiography/rx-simplecnn-20260515-...

Modelo seleccionado

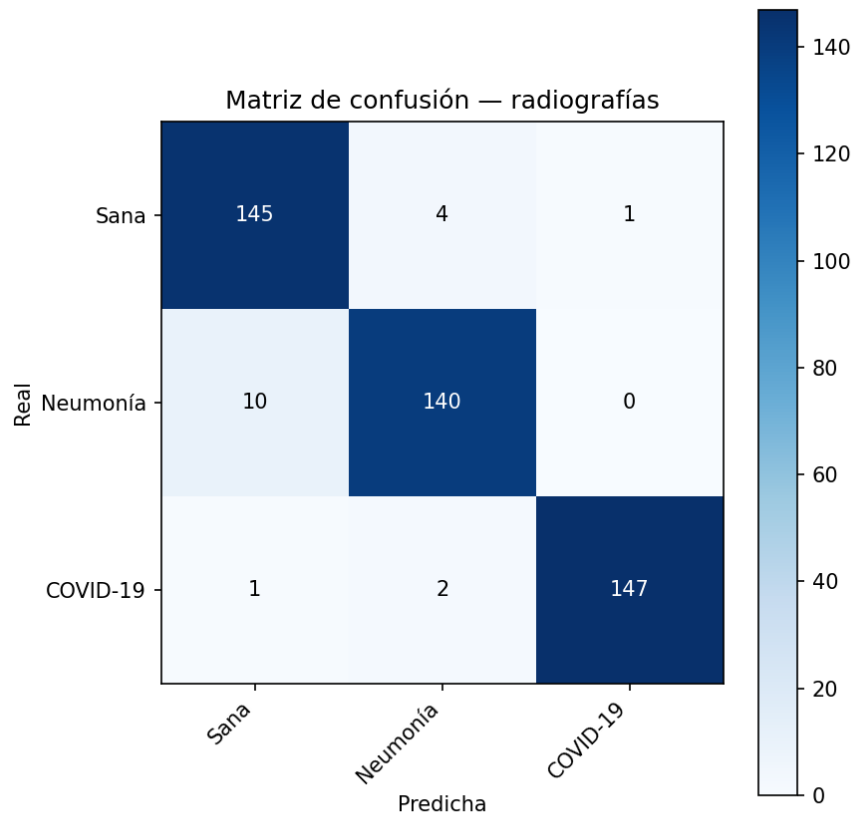
El modelo seleccionado por equilibrio clínico es:

- **Versión:** `rx-efficientnetb0-20260515-2b92eec9`
- **Backbone:** `efficientnet_b0`
- **Accuracy:** `0.9600`
- **F1 macro:** `0.9601`
- **Recall COVID-19:** `0.9800`
- **Recall Neumonía:** `0.9333`

Se selecciona porque combina buen rendimiento global con alta sensibilidad en las clases clínicamente más relevantes.

Informe `comparison.md`

Ejemplo: Modelo resnet18



Errores

#	Clase real	Clase predicha	Casos
1	Neumonía	Sana	10
2	Sana	Neumonía	4
3	COVID-19	Neumonía	2
4	Sana	COVID-19	1
5	COVID-19	Sana	1

Falso negativo COVID-19

Una radiografía de COVID-19 clasificada como Sana o Neumonía tiene riesgo epidemiológico alto. Puede provocar falta de aislamiento, exposición del personal sanitario y contagios intrahospitalarios. Este es uno de los errores más graves del sistema.

COVID-19 confundido con neumonía

El riesgo es medio-alto. Parte del manejo respiratorio puede ser similar, pero se pierde el componente epidemiológico: aislamiento, trazabilidad de contactos y protocolos COVID.

Neumonía clasificada como sana

Implica retraso diagnóstico y terapéutico, especialmente problemático en pacientes vulnerables. Puede retrasar antibiótico, seguimiento o derivación.

Sana clasificada como patológica

Es menos crítico que un falso negativo, pero genera sobrecarga asistencial, pruebas innecesarias, ansiedad del paciente y coste operativo.

El recall COVID-19 (0.980) es razonable para un prototipo, aunque seguiría requiriendo validación clínica externa.

Limitaciones reales

- Dataset público con sesgos de adquisición, procedencia y calidad.
- No existe validación externa con datos hospitalarios propios.
- La clase Neumonía agrupa Viral Pneumonia y Lung_Opacity, simplificando una realidad clínica más compleja.
- El modelo puede aprender artefactos del dataset en lugar de patrones radiológicos generalizables.
- El sistema no sustituye criterio médico ni constituye dispositivo médico certificado.

El modelo es válido como módulo académico de Deep Learning integrado en una infraestructura Big Data hospitalaria. Para producción real harían falta más datos, validación multicéntrica, revisión radiológica experta, calibración de probabilidades y certificación regulatoria.

4.6. Reentrenamiento del modelo

El modelo de radiografías no se reentrena automáticamente cuando se sube una nueva imagen al dashboard. En esa situación, el sistema solo ejecuta inferencia: recibe la imagen, la preprocesa y la clasifica usando el artefacto `model.pt` ya entrenado. El entrenamiento se realiza de forma offline y reproducible mediante scripts Docker (`prepare_data.py`, `train.py`, `evaluate.py`, `compare_models.py`). Dask se utiliza para el procesamiento escalable del pipeline batch tabular, no para entrenar las redes neuronales de radiografías. Como mejora futura, podríamos implementar un flujo MLOps de reentrenamiento periódico o condicionado a nuevos datos validados.

4.7. Control de overfitting

Para reducir el riesgo de overfitting, separamos el dataset en train, validation y test, manteniendo una distribución estratificada por clase. Durante el entrenamiento se monitorizó el rendimiento en validación mediante val_f1_macro y se guardaron históricos (history.csv y history.json) para revisar la evolución del aprendizaje. También aplicamos data augmentation moderado y transfer learning para mejorar la generalización sin alterar de forma agresiva los patrones radiológicos. La evaluación final se hizo sobre test y se complementó con matrices de confusión y análisis crítico clínico. Aun así, reconocemos como limitación que no existe validación externa hospitalaria, por lo que no puede afirmarse ausencia completa de overfitting clínico.

5. Automatizaciones

5.1 Automatizaciones implementadas

Implementamos el servicio automation como un proceso separado. Su responsabilidad es revisar periódicamente la colección system_events de MongoDB y detectar eventos con nivel warning o error. Cuando encuentra un evento relevante, crea una alerta deduplicada en la colección alerts. Esta alerta funciona como notificación simulada y puede ser revisada desde MongoDB, Mongo Express o dashboard.

Automatización	Entrada	Salida	Objetivo
Escaneo de eventos warning/error	MongoDB system_events	MongoDB alerts	Detectar anomalías operativas.
Deduplicación de alertas	dedup_key	Alerta única actualizada	Evitar alertas repetidas.
Logs de automation	stdout	Loki/Promtail	Auditar el servicio de automatización.

5.2 Integración dentro del sistema

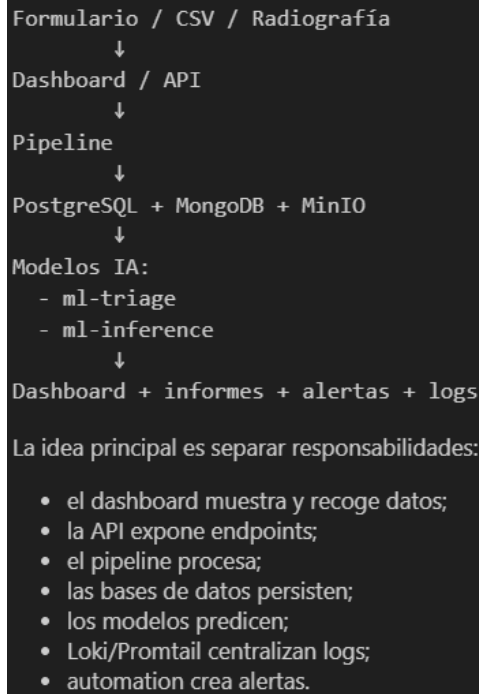
automation se integra sin bloquear el pipeline. El pipeline emite eventos; automation los consume de forma asíncrona. Esto permite desacoplar procesamiento y alertas: si automation falla, el pipeline puede seguir funcionando, y cuando se recupere podrá volver a escanear eventos recientes.

6. Integraciones

6.1 Flujo completo de datos

El flujo completo empieza con una ficha individual, un CSV batch o una radiografía. En el caso tabular, dashboard/API envían datos al pipeline; el pipeline valida, transforma y persiste; después llama a ml-triage; las predicciones se guardan en MongoDB y se muestran en el dashboard. En el caso de radiografías, el dashboard envía la imagen a ml-inference y recibe la clase y probabilidades.

1. Flujo lógico end-to-end



6.2 Conexión entre módulos

Origen	Destino	Tipo de conexión	Uso
dashboard	api	HTTP	Gestión de pacientes y batch runs.
api	pipeline	CLI/servicio interno	Lanzar procesamiento.
pipeline	minio	S3-compatible	Leer/guardar datos raw.
pipeline	postgres	SQL	Guardar pacientes e ingresos.
pipeline	mongodb	MongoDB	Guardar eventos, rechazos y predicciones.
pipeline	ml-triage	HTTP	Inferencia tabular.
dashboard	ml-inference	HTTP	Clasificación de radiografías.
promtail	loki	HTTP push	Enviar logs.
automation	mongodb	MongoDB	Leer eventos y crear alertas.

7. Diario de desarrollo con IA

7.1 Herramientas utilizadas

Durante el proyecto utilizamos herramientas de desarrollo asistido por IA como apoyo integral al flujo de trabajo. Claude Code se utilizó como pair programmer dentro del repositorio, especialmente para escribir y modificar código. ChatGPT se utilizó como apoyo de arquitectura, depuración, generación de comandos, redacción técnica, revisión contra requisitos y preparación de notebooks/documentación.

Herramienta	Uso principal	Justificación
Claude Code	Edición y generación de código en el repositorio.	Adecuado para iterar sobre archivos reales.
ChatGPT	Diseño, depuración, documentación, comandos y defensa.	Adecuado para razonar sobre arquitectura y explicar decisiones.

7.2 Ejemplos de prompts

- “Quiero un dashboard profesional con menú en header, pacientes, ficha desplegable, PDF y paleta hospitalaria.”
- “Que es el ETL batch? ¿Podemos subir un dataset con varias fichas?”
- “Arreglamos Loki porque health está OK pero la query devuelve result vacío.”
- “Hazme README del módulo ml-inference explicando dataset, modelos, métricas y análisis clínico.”
- “Comparar CNN simple, ResNet18 y EfficientNet-B0 y justificar cuál es mejor.”

7.3 Problemas encontrados y soluciones

Problema	Solución
Healthcheck de MongoDB incompatible con imagen actual.	Se corrigió usando herramientas compatibles con la imagen.
YAML interpretaba no como booleano False.	Se corrigió quoting de valores.
Promtail recogía logs de otros proyectos Docker.	Se ajustó configuración y se verificaron labels reales.
Loki healthy pero query vacía.	Se depuró LogQL y rangos start/end hasta obtener pipeline.run.end.
Dataset de radiografías incluía masks/.	Se corrigió prepare_data.py para leer solo images/.
Automation fallaba por conflicto updated_at/last_seen_at.	Se separaron campos de insert y update.
GitHub rechazó ZIP grande.	Se ajustó .gitignore y se verificó que no hubiera archivos >95 MB.

8. Justificaciones técnicas

8.1 Decisiones tomadas

Decisión	Justificación
Docker Compose	Permite levantar toda la infraestructura con un solo comando.
MinIO en vez de S3 real	Evita credenciales externas y permite demo local reproducible.
PostgreSQL + MongoDB + MinIO	Cada tecnología cubre una necesidad distinta: relacional, documental y raw.
Dask	Framework escalable ligero y compatible con pandas.
FastAPI	API rápida, tipada y adecuada para microservicios Python.
Streamlit	Permite dashboard clínico rápido y demostrable.
EfficientNet-B0	Mejor equilibrio clínico en métricas de radiografías.
Loki/Promtail	Logging centralizado integrado con Docker.
MongoDB alerts	Simula notificaciones operativas sin depender de email real.

8.2 Alternativas consideradas

Alternativa	Motivo para no escogerla
AWS S3 real	Complica la evaluación local por credenciales y conectividad.
Apache Airflow	Útil como orquestador DAG, pero el requisito pedía procesamiento escalable; Dask era más adecuado y ligero.
Spark	Más pesado para entorno local de práctica.
Solo CNN simple	Rendimiento inferior frente a transfer learning.
Solo accuracy como métrica	Insuficiente en contexto sanitario; importan falsos negativos.

9. Reflexión crítica

9.1 Limitaciones del sistema

- El sistema es académico y no está validado clínicamente.
- Los modelos tabulares se entrenan con datos sintéticos.
- Las radiografías proceden de un dataset público, no de un hospital local.
- El modelo de radiografías solo clasifica tres clases y fuerza siempre una salida.
- No se implementa detector de imagen fuera de distribución.
- No hay calibración formal de probabilidades.
- No hay validación por subgrupos demográficos.
- No hay integración real con HIS/RIS/PACS hospitalario.
- No se ha implementado Grad-CAM ni explicabilidad visual sobre radiografías.

9.2 Posibles mejoras

- Añadir detector de calidad y tipo de imagen antes de ml-inference.
- Añadir clase “imagen no válida” o “fuera de distribución”.
- Implementar Grad-CAM para visualizar regiones relevantes de la radiografía.
- Añadir validación externa con otro dataset hospitalario.
- Calibrar probabilidades de modelos clínicos.
- Añadir autenticación y roles de usuario.
- Añadir trazabilidad legal de informes y acciones.
- Ampliar automation a email o notificaciones reales.

9.3 Aplicación en un entorno real

En un entorno real, este sistema debería utilizarse únicamente como apoyo a la decisión. La responsabilidad final debe permanecer en el personal sanitario. Para producción serían necesarias validación clínica, revisión legal, ciberseguridad, control de accesos, auditoría de datos y monitorización post-despliegue.

9.4 Nota informal opcional

El usuario propuso incluir una broma sobre Psyduck como “el mejor Pokémon”. No la hemos integrado en el cuerpo técnico principal porque podría romper el tono académico de la memoria. Si se desea mantenerla, recomendamos ubicarla en una nota informal fuera de la entrega oficial o en un anexo no evaluable.

10. Consideraciones éticas y legales

10.1 Sesgos en datos y modelos

Los modelos pueden heredar sesgos de los datos. En radiografías, el dataset público puede no representar todos los hospitales, equipos, países, edades o grupos clínicos. En datos tabulares, los síntomas auto-reportados pueden variar según edad, idioma, nivel de alfabetización sanitaria o percepción subjetiva del dolor. Por ello analizamos métricas por clase y no solo accuracy.

10.2 Privacidad y protección de datos

En un entorno real, los datos sanitarios se consideran especialmente sensibles. En esta práctica usamos datos sintéticos y públicos para evitar tratar datos identificables reales. Además, mantenemos .env fuera de Git,

evitamos subir datasets pesados o privados y usamos pseudo_id en lugar de identificadores reales. En producción haría falta una evaluación de impacto, control de accesos, cifrado, auditoría y revisión legal.

10.3 Riesgos del sistema

- Falso negativo COVID-19: riesgo de no aislar a un paciente contagioso.
- Neumonía clasificada como sana: retraso terapéutico.
- Triage alto clasificado como bajo: demora asistencial.
- Falso positivo: sobrecarga asistencial, ansiedad y pruebas innecesarias.
- Sesgo de automatización: el usuario puede confiar demasiado en el modelo.

Mitigamos estos riesgos mostrando probabilidades, baja confianza, análisis crítico, matrices de confusión y mensajes de apoyo a la decisión. El sistema no debe usarse como diagnóstico autónomo.

11. Capturas y evidencias recomendadas

Para hacer la memoria más visual, recomendamos insertar capturas en los espacios marcados. Las más importantes son:

Captura	Dónde insertarla	Qué demuestra
Estructura del código	Descripción del problema / arquitectura	Organización del repositorio y separación de módulos.
docker compose ps	Arquitectura del sistema	Servicios levantados y containerización.
Dashboard pacientes	Visualización	Resultados clínicos navegables.
Pestaña radiografías	Modelos IA	Inferencia de imagen integrada en dashboard.
Ranking modelos	Resultados IA	Comparación CNN/ResNet18/EfficientNet-B0.
Matriz de confusión	Evaluación IA	Análisis de errores del modelo.
Loki query	Monitorización	Logging centralizado funcionando.
MongoDB alerts	Automatizaciones	Alertas generadas por automation.
Notebook overview	Comunicación de resultados	Explicación reproducible del sistema.

laSalle Health Center

Modelo de Soporte Hospitalario — Sistema de Soporte Hospitalario

Pacientes

Traje

Radiografías

Casos

Clasificación de radiografías

Modelo de Soporte Hospitalario — Sistema de Soporte Hospitalario / COVID-19

Sube una radiografía de tórax

Drag and drop file here
Limit: 20MB por foto - PNG, JPG, JPEG

Browse files

COVID-19.png (31 KB)

Radiografía cargada

Clasificar radiografía

Resultado: COVID-19

Sana 0.0%	Neumonía 1.7%	COVID-19 98.3%
--------------	------------------	-------------------

Alerta COVID-19: probabilidad superior al umbral configurado.

Resumen completo del modelo

```

{
  "probabilidad_sana": 0.0,
  "probabilidad_neumonia": {
    "neumonia": 0.017,
    "neumonia_covid": 0.983
  },
  "resultado": "COVID-19"
}

```


laSalle Health Center

Sistema de Soporte Hospitalario — Apoya a la decisión clínica

API conectada

PacientesTriajeRadiografíasEstado

Triaje

Formulario individualLote CSV

Ficha del paciente

Re llena los campos y pulsa **Clasificar** para que el sistema calcule el nivel de urgencia.

Datos básicos

Edad
66

Sexo
Femenino

Peso (kg)
70

Altura (cm)
162

Antecedentes

Enfermedades crónicas
HipertensiónArterioesclerosis

Fumador
No

Embarazo
No

Síntomas actuales

Motivo de consulta
Dolor torácico

Duración
Menos de 24 h

Intensidad del dolor (0-10)
8

Fiebre subjetiva
No

Dificultad respiratoria
Leve

Tos
No

Contacto reciente con caso COVID-19
No

Clasificar

PACIENTE REGISTRADO — PAT-000506

Alta

Modelo: tri-20260514-60eff3c2

Alta
98.8%

Media
1.1%

Baja
0.1%

SOSPECHA

Sospecha cardiopatía aguda (100%)

Sospecha orientativa basada en síntomas auto-reportados. No sustituye valoración médica. Modelo: dls-20260514-8c9a3874

Registro de ejecución (eventos del pipeline online)

Eventos
4

Estado
Terminado

Transcurrido
0 s

09:52:35 pipeline.online.start Procesando ficha online

09:52:36 pipeline.online.raw_persisted Ficha raw persistida en s3://hospital-raw@http://minio:9000/online/2026/05/18/online-f8f6d78dba31.json

09:52:37 pipeline.loading.done Cargados 1 pacientes y 1 ingresos

09:52:39 pipeline.online.end Ficha PAT-000506 procesada (completed)

Resumen final

pseudo_id
PAT-000506

ml_status
completed

raw_key
online/2026/05/18/o...

Polling detenido.

Este resultado es apoyo a la decisión, no un diagnóstico. El personal sanitario revisa siempre la clasificación.

Alertas en MongoDB

```
{
  _id: ObjectId('6a06edfd4f9b4c3f403b4800'),
  dedup_key: 'pipeline.rejects.persisted:run-20260515-c4a74267:warn',
  correlation_id: 'run-20260515-c4a74267',
  created_at: ISODate('2026-05-15T09:57:17.568Z'),
  first_seen_at: ISODate('2026-05-15T09:57:17.568Z'),
  last_message: '4 rechazos persistidos en ingestion_rejects',
  last_payload: { count: 4 },
  last_seen_at: ISODate('2026-05-15T10:06:47.790Z'),
  message: '4 rechazos persistidos en ingestion_rejects',
  notification_channel: 'mongo_dashboard',
  notification_status: 'simulated',
  payload: { count: 4 },
  severity: 'warning',
  source_event: 'pipeline.rejects.persisted',
  source_event_id: '6a06edeb8fdcdbba3b394384',
  source_service: 'pipeline',
  source_timestamp: ISODate('2026-05-15T09:56:59.463Z'),
  status: 'open',
  updated_at: ISODate('2026-05-15T10:06:47.790Z')
},
{
  _id: ObjectId('6a06edfd4f9b4c3f403b47ff'),
  dedup_key: 'pipeline.validation.done:run-20260515-c4a74267:warn',
  correlation_id: 'run-20260515-c4a74267',
  created_at: ISODate('2026-05-15T09:57:17.563Z'),
  first_seen_at: ISODate('2026-05-15T09:57:17.563Z'),
  last_message: 'Validación: 1 válidos, 4 rechazados',
  last_payload: { valid: 1, rejected: 4 },
  last_seen_at: ISODate('2026-05-15T10:06:47.787Z'),
  message: 'Validación: 1 válidos, 4 rechazados',
  notification_channel: 'mongo_dashboard',
  notification_status: 'simulated'
}
```

Warnings/errors que generan alertas

```
{
  _id: ObjectId('6a06edeb8fdcdbba3b394384'),
  timestamp: ISODate('2026-05-15T09:56:59.463Z'),
  service: 'pipeline',
  event: 'pipeline.rejects.persisted',
  level: 'warning',
  correlation_id: 'run-20260515-c4a74267',
  message: '4 rechazos persistidos en ingestion_rejects',
  payload: { count: 4 }
},
{
  _id: ObjectId('6a06edeb8fdcdbba3b39437d'),
  timestamp: ISODate('2026-05-15T09:56:59.345Z'),
  service: 'pipeline',
  event: 'pipeline.validation.done',
  level: 'warning',
  correlation_id: 'run-20260515-c4a74267',
  message: 'Validación: 1 válidos, 4 rechazados',
  payload: { valid: 1, rejected: 4 }
},
{
  _id: ObjectId('6a06edb603bd0026b07c14d6'),
  timestamp: ISODate('2026-05-15T09:56:06.108Z'),
  service: 'pipeline',
  event: 'pipeline.rejects.persisted',
  level: 'warning',
  correlation_id: 'run-20260515-0d2f6e8a',
  message: '4 rechazos persistidos en ingestion_rejects',
  payload: { count: 4 }
},
{
  _id: ObjectId('6a06edb503bd0026b07c14cf'),
  timestamp: ISODate('2026-05-15T09:56:05.832Z'),
  service: 'pipeline',
  event: 'pipeline.validation.done',
  level: 'warning',
  correlation_id: 'run-20260515-0d2f6e8a',
  message: 'Validación: 1 válidos, 4 rechazados',
  payload: { valid: 1, rejected: 4 }
}
```

Query Loki sobre automation

Time	Service	Container	Log
----	-----	-----	---
18/05/2026 11:41:07	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:41:07,306 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0
18/05/2026 11:40:37	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:40:37,307 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0
18/05/2026 11:40:07	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:40:07,279 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0
18/05/2026 11:39:37	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:39:37,303 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0
18/05/2026 11:39:07	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:39:07,302 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0
18/05/2026 11:38:37	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:38:37,304 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0
18/05/2026 11:38:07	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:38:07,304 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0
18/05/2026 11:37:37	automation	practica_hospital_ariadnapascualpolballarin-automation-1	2026-05-18 09:37:37,305 [INFO] automation: Escaneo completado: events_scanned=0 alerts_inserted=0 alerts_existing=0

Las alertas operativas se persisten en MongoDB en la colección alerts, generadas a partir de eventos warning/error almacenados en system_events. Loki/Promtail centralizan los logs técnicos de los servicios, y en la captura se observa el servicio automation ejecutando escaneos periódicos.

12. Conclusiones

Hemos desarrollado una solución integral que simula un sistema real de soporte hospitalario basado en IA y Big Data. La solución no se limita a entrenar modelos, sino que integra infraestructura containerizada, pipeline escalable, almacenamiento combinado, modelos tabulares, Deep Learning de radiografías, API, dashboard, monitorización, validación de calidad y alertas.

El resultado demuestra que el sistema puede ingerir datos, limpiarlos, transformarlos, analizarlos, servirlos y monitorizarlos. También documentamos limitaciones, riesgos éticos y decisiones técnicas. La conclusión principal es que la IA aporta valor como apoyo a la decisión, pero no debe sustituir el criterio clínico ni utilizarse en producción sin validación formal.

13. Referencias y documentación de apoyo

- European Commission — AI Act enters into force: referencia para sistemas de IA de alto riesgo y supervisión humana en contextos sensibles.
- European Data Protection Board — GDPR Article 9: referencia sobre categorías especiales de datos personales, incluyendo datos de salud.
- Documentación oficial de PyTorch y Torchvision: arquitecturas ResNet18 y EfficientNet-B0.
- Documentación oficial de Dask: scheduler, workers y procesamiento escalable.
- Documentación oficial de Grafana Loki y Promtail: logging centralizado.
- Documentación del repositorio: README.md, CHANGELOG.md, specs/SDD-*.md, notebooks/ y docs/diario-ia/.

Anexo A — Lista rápida de comandos de verificación

Objetivo	Comando
Levantar sistema	<code>docker compose --env-file .env up -d --remove-orphans</code>
Ver servicios	<code>docker compose ps</code>
Health API	<code>curl.exe http://localhost:8000/health</code>
Health ml-inference	<code>docker compose exec ml-inference curl -s http://localhost:8001/healthz</code>
Loki ready	<code>curl.exe -s http://localhost:3100/ready</code>
Eventos Mongo	<code>db.system_events.find().sort({timestamp:-1}).limit(10).pretty()</code>
Alertas Mongo	<code>db.alerts.find().sort({created_at:-1}).limit(10).pretty()</code>
Predicción radiografía	<code>curl.exe -X POST http://localhost:8001/predict -F "file=@ruta.png"</code>